

ACTIVIDAD SEMANAL 05

En este caso, debido por el tiempo que tuve lo realicé solo nuevamente, más que todo por el tiempo dado, ya que no pude ponerme de acuerdo con mis compañeros, por temas del horario en mi trabajo que se complicaba un poco.

Ahora pasando con el código, lo cambie un poco ya que debido a que por el tiempo, no logro entrar a la sesiones, no logro comprender bien los temas y decidí hacerlo más sencillo, sin usar parámetros ya que todavía no domino bien el lenguaje de Python, ya que con las videoconferencia no se logra entender al 100% debido que es por partes y no es toda la clase grabada.

ORDENAMIENTO POR SELECCIÓN:

Comenzamos con lo básico, definimos nuestra lista base con los valores desordenados y los mostramos mediante consola, para que el que visualice el código no se confunde de donde salen los valores, que se muestran.

```
numeros = [2, 8, 5, 3, 9, 4, 1]
print(f"Lista original: {numeros}")
print(" ")
```

Posteriormente se crea una copia independiente de la lista original, así los cambios de este método no modifican la lista base para la siguiente codificación. Y también creamos una variable *n* en donde se guarda el tamaño de la lista, que en este caso son 7 elementos.

```
lista = list(numeros)
n = len(lista)
```

Luego, ya comenzamos con el *Ordenamiento por Selección*.

Se comienza con un *for* que recorre todas las posiciones de la lista una por una hasta la penultima, luego creamos *pos_min* para asumir temporalmente que es donde se encuentra el valor mas pequeño de la posicion actual *i*.

```
print("-- ORDENAMIENTO POR SELECCION --")

for i in range(n - 1):
    pos_min = i
    for j in range(i + 1, n):
        if lista[j] < lista[pos_min]:
            pos_min = j
    lista[i], lista[pos_min] = lista[pos_min], lista[i]
    print(f"Paso {i + 1}: {lista}")
print(" ")
```

Mediante el segundo *for*, se revisa el resto de la lista buscando si es que se encuentra un valor menor al actual. Con el *if*, comparamos cada elemento con el mínimo conocido, y si se encuentra uno mas pequeño se guarda en la variable *pos_min*. Luego intercambiamos el elemento de la posicion [*i*] con el valor mínimo encontrado en la variable *pos_min*.

Y lo que se busca con el *print(...)* es que se muestre como se va intercambiando cada posicion para que se pueda comprender mas a fondo el método.

ORDENAMIENTO POR BURBUJA:

Igual que lo anterior se comienza creando una copia independiente de la lista original y la variable n en donde se guarda el tamaño de nuestra lista.

Comenzamos con un *for*, que controla el numero de pasada completadas sobre la lista. En el segundo *for* comparamos elementos, con cada pasada i , el

elemento mas grande se queda al final, por lo que no hace falta volver a revisar los últimos i elementos. Luego comparamos los elementos, si es que el de la izquierda es mayor que el de la derecha los intercambia hasta tenerlos todos completamente ordenados.

Igual que lo anterior lo que se busca con el *print(...)* es que se luzca paso a paso para poder comprender mejor.

```
lista = list(numeros)
n = len(lista)
print("-- ORDENAMIENTO POR BURBUJA --")

for i in range(n):
    for j in range(0, n - 1 - i):
        if lista[j] > lista[j + 1]:
            lista[j], lista[j + 1] = lista[j + 1], lista[j]
            print(f"Paso {i + 1}: {lista}")
print(" ")
```

ORDENAMIENTO POR INSERCIÓN:

Igual que los anterior ordenamientos se comienza creando una copia limpia de la lista original, en este caso ya no creamos la variable n .

Con nuestro primer *for*, se ve las pasadas de la lista, pero ahora comenzamos por el segundo elemento $i[1]$, porque el elemento $i[0]$ se considera ya "ordenado". Se crea la variable *clave* y j , donde guardamos el numero que se va a compara e insertar en su posicion correcta. En la variable j se apunta al índice anterior para empezar a comparar hacia la izquierda.

Seguimos con un *while*, donde se define que mientras existan elementos a la izquierda y sean mayores que la variable *clave*, los desplaza a un lugar hacia la derecha. Luego en la variable j se retrocede una posicion hacia la izquierda para seguir comparando los elementos. En $lista[j + 1] = clave$, se inserta *clave* en el espacio libre que queda tras los desplazamientos y así asegurar que esta todo correctamente ordenado y no tenemos espacios en blanco o vacíos.

Igual que lo anterior lo que se busca con el *print(...)* es que se luzca paso a paso para poder comprender mejor.

```
lista = list(numeros)
print("-- ORDENAMIENTO POR INSERCIÓN --")

for i in range(1, len(lista)):
    clave = lista[i]
    j = i - 1
    while j >= 0 and lista[j] > clave:
        lista[j + 1] = lista[j]
        j -= 1
    lista[j + 1] = clave
    print(f"Paso {i}: {lista}")
print(" ")
```